



ระบบจำลองมือตัวเลขไทย ๒๖-๓๐

By Folder

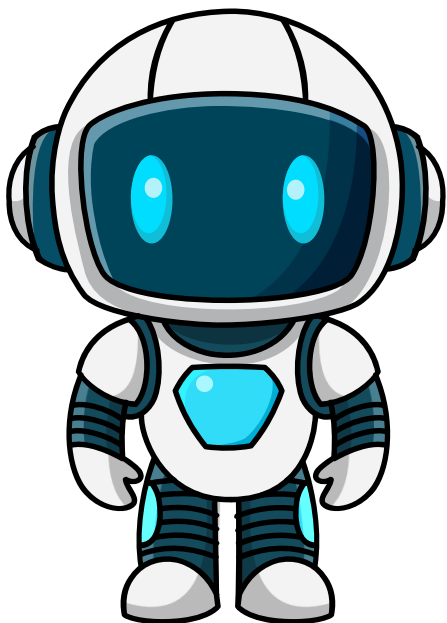


Introduction

ระบบรู้จำลายมือตัวเลขไทย ๒๖-๓๐ พัฒนารุ่นขึ้นเพื่อจำแนกตัวเลขที่มีความซับซ้อนด้วยโมเดล CRNN มีความแม่นยำสูงถึง 95% (F1-score 0.95) โดยผ่านการทดสอบด้วย 5-Fold Cross-validation อีกทั้งระบบมีกระบวนการ Preprocessing อัตโนมัติ (Denoise, Threshold, Centering) เพื่อเพิ่มความเสถียร รองรับการใช้งานแบบ Real-time ผ่านเว็บแอปพลิเคชันที่ผู้ใช้สามารถวาดและทำนายผลได้ทันทีผ่าน API พร้อมระบบ Model Management ให้ผู้ดูแลระบบสามารถอัปเดตโมเดลใหม่ได้อย่างสะดวกและมีประสิทธิภาพ

URL Link S๑UU : <https://peeradon-bu-thai-digit-recognizer.hf.space>

URL Github : https://github.com/parewaa/Project_CS462.git



Algorithm CRNN — โครงสร้างและ Data Flow

Algorithm ที่เลือก

CRNN

Convolutional Recurrent Neural Network

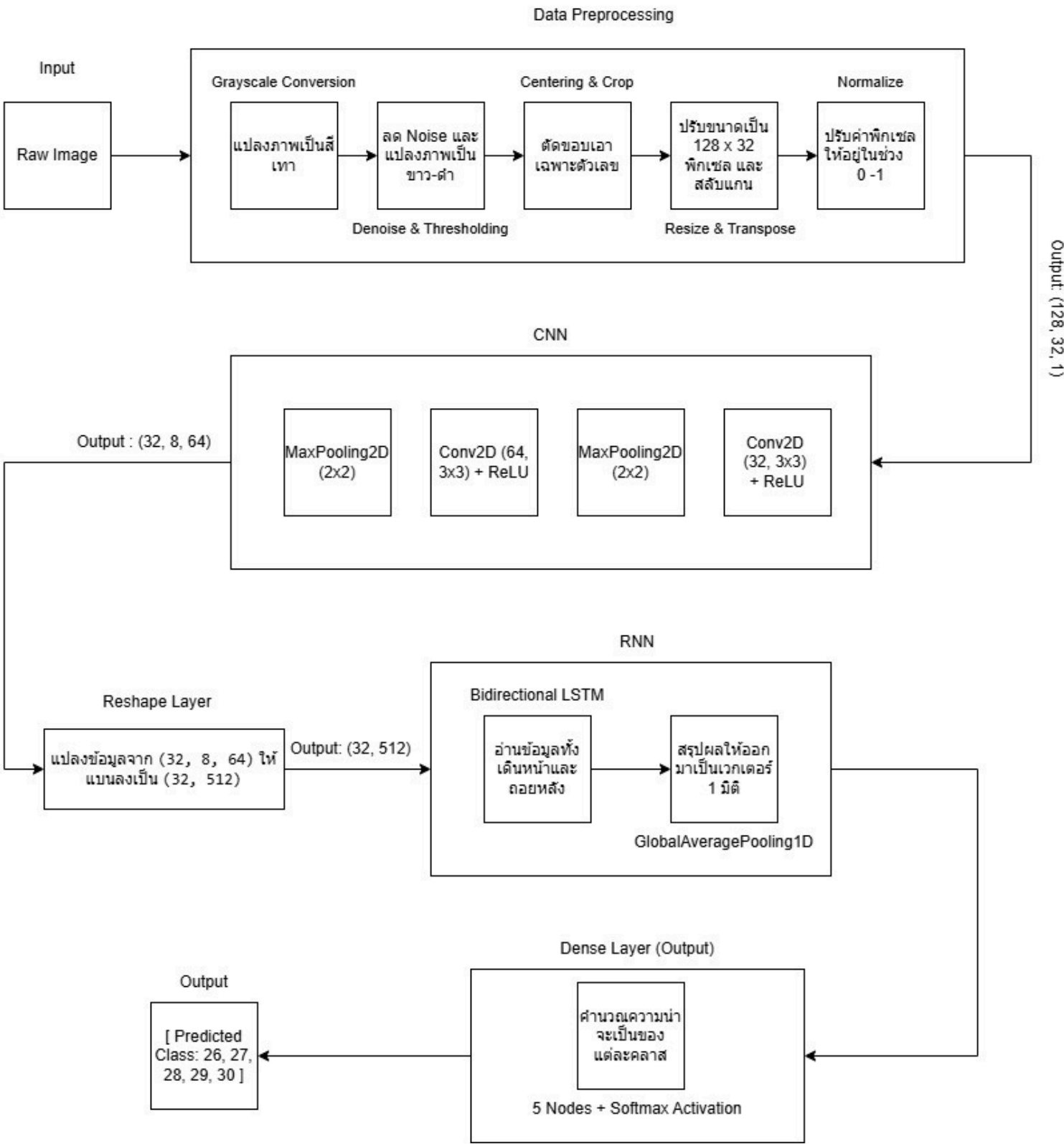
Hybrid Architecture

CNN ทำหน้าที่เป็น "ดวงตา" สกัดฟีเจอร์สำคัญ
RNN ทำหน้าที่เป็น "สมอง" วิเคราะห์ความต่อเนื่องของลายเส้น

ทนทานต่อความยาวและขนาด
ตัวเลขที่ไม่แน่นอน

ประสิทธิภาพสูงในชุดข้อมูลที่ซับซ้อน

Architecture & Data Flow



การ Train โมเดล — Dataset & Classification

Classify อะไร

5 คลาส — เลขไทย ๒๖ · ๒๗ · ๒๘ · ๒๙ · ๓๐

ลายมือเขียนบน Canvas ใน Browser

รูปแบบ: PNG · RGBA · 800×800 px

วิธีเก็บ Dataset

- เขียนตัวเลขบน Canvas ใน Browser
- บันทึกเป็น PNG โดยอัตโนมัติ
- จัดเก็บแยกโฟลเดอร์ตามคลาส (26–30)

626

รูปทั้งหมด

~125

รูป/คลาส

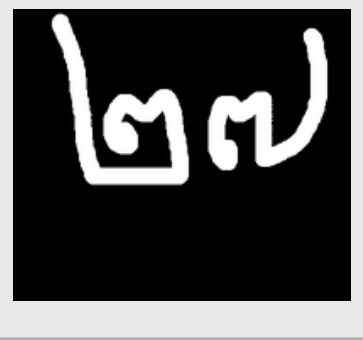
5

คลาส

ตัวอย่าง Dataset — คลาสละ 1 รูป



๒๖ (26)



๒๗ (27)



๒๘ (28)



๒๙ (29)



๓๐ (30)

Preprocessing Pipeline

1

Grayscale

ใช้ cv2.imread เพื่อโหลดภาพในรูปแบบขาว-ดำ

2

Denoising

ใช้ cv2.fastNlMeansDenoising เพื่อกำจัด Noise ในภาพออก

3

Thresholding

ใช้เทคนิค Otsu's Thresholding เพื่อแปลงภาพจาก Grayscale ให้พื้นหลังเป็นสีดำและตัวเลขเป็นสีขาว

4

Centering & Crop

ใช้ cv2.findNonZero ทำการตัดเอาเฉพาะส่วนที่มีตัวเลข

5

Resizing

ปรับขนาดภาพใหม่ให้มีความกว้าง 128 พิกเซล และความสูง 32 พิกเซล

6

Axis Transpose

เปลี่ยนแกนภาพเป็น (128, 32) จาก (32, 128) เพื่อให้เหมาะสม RNN (LSTM) ที่อ่านข้อมูลตามแนวนอน

7

Normalization

แปลงค่าพิกเซลจากระดับ 0-255 ให้เป็นค่าทศนิยมในช่วง 0.0 ถึง 1.0

8

Expand Dimensions

ทำให้ภาพอยู่ในรูปแบบ (128, 32, 1) ซึ่งเป็น Input Shape ที่ Keras

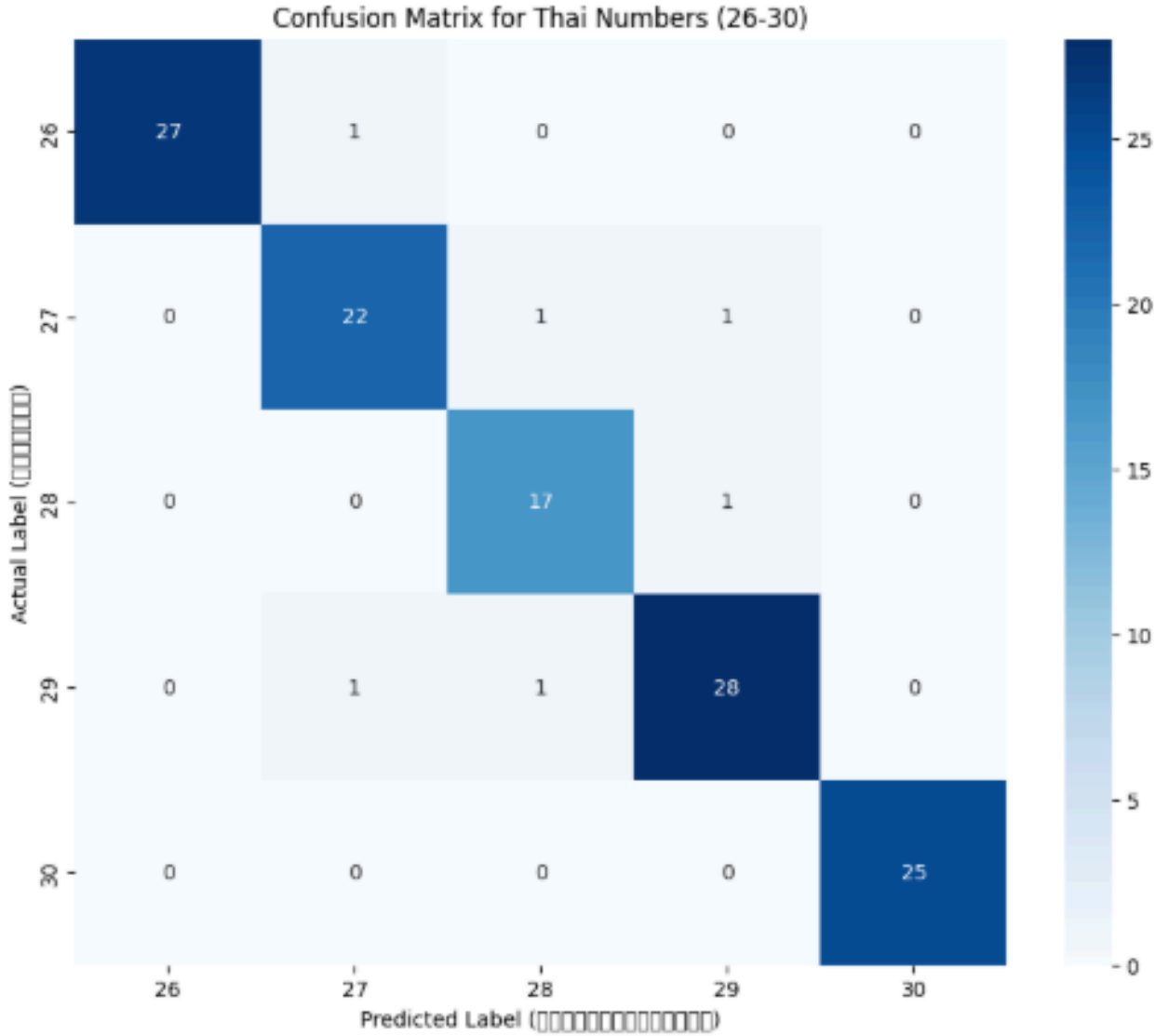
ผลการทดสอบ & วิเคราะห์ประสิทธิภาพ

	precision	recall	f1-score
26	1.00	0.96	0.98
27	0.92	0.92	0.92
28	0.89	0.94	0.92
29	0.93	0.93	0.93
30	1.00	1.00	1.00
accuracy			0.95
macro avg	0.95	0.95	0.95
weighted avg	0.95	0.95	0.95

วิเคราะห์ประสิทธิภาพ

- Accuracy : โมเดลสามารถทายตัวเลขไทยได้ถูกต้องแม่นยำสูงถึง 95% จากข้อมูลทดสอบทั้งหมด
- Precision : เมื่อโมเดลทำนายว่าเป็นเลขหนึ่งๆ (เช่น เลข ๒๗) ผลปรากฏว่าเป็นเลขนั้นจริงๆ สูงแสดงว่าโมเดลทายมีว้น้อยมาก
- Recall : โมเดลสามารถตรวจจับและดึงตัวเลขที่มีอยู่จริงในชุดข้อมูลมาตอบได้ถูกต้องถึง 95% แสดงว่าโมเดลไม่มองข้ามตัวเลขจริงไป
- F1-Score : เนื่องจากค่า Precision และ Recall สูงพอๆกันและสมดุลกัน ค่า F1 ซึ่งเป็นค่าเฉลี่ยของทั้งคู่จึงสูงตาม ยืนยันได้ว่าโมเดลมีประสิทธิภาพที่เสถียร ไม่ได้เก่งแค่ด้านใดด้านหนึ่ง

Confusion Matrix



Cross-validation

ความแม่นยำไม่ได้มาจากการสุ่มแบ่งข้อมูลเพียงครั้งเดียว แต่ผ่านการทำ 5-Fold Cross-validation มาแล้ว ทำให้มั่นใจได้ว่าหากเปลี่ยนชุดข้อมูลเป็นลายมือใหม่ๆ โมเดลจะยังรักษามาตรฐานความแม่นยำที่ระดับ 90%+ ไว้ได้

เพิ่มเติม - ผลการทดสอบ & วิเคราะห์ประสิทธิภาพ ของโมเดล CNN

81.91%

Test Accuracy

0.5754

Test Loss

0.82

Macro F1-Score

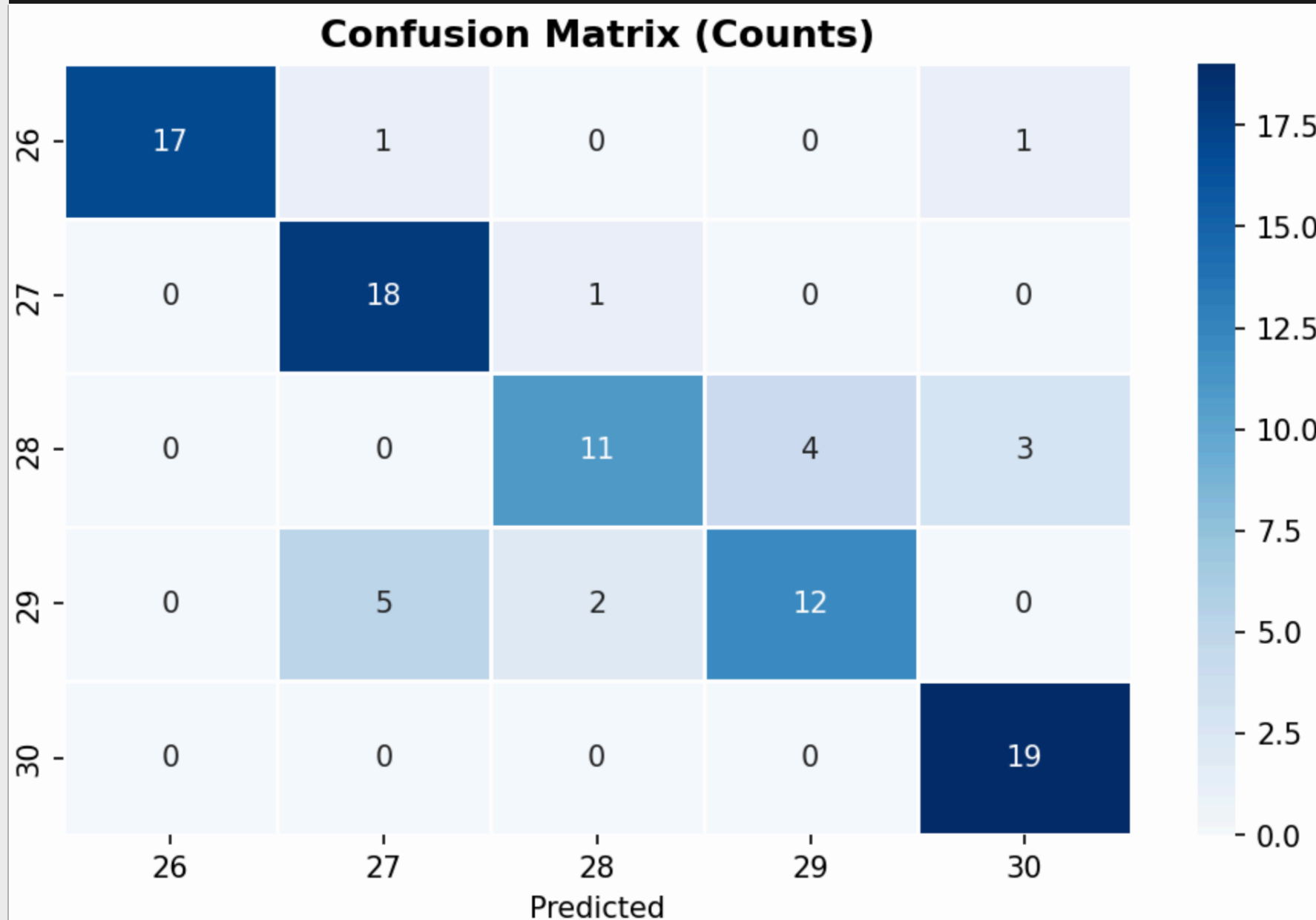
~0.97

Macro AUC

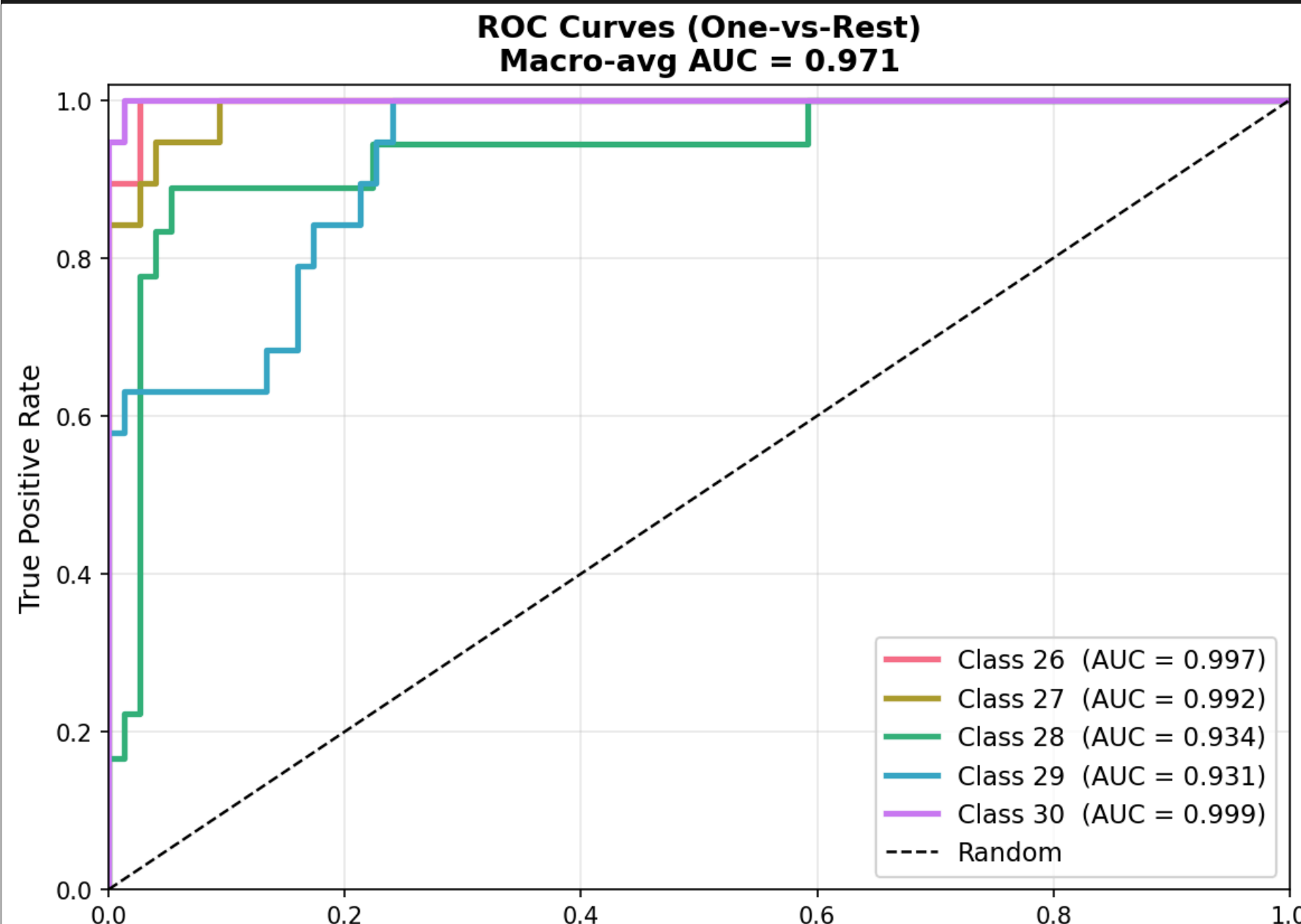
94

Test Samples

Confusion Matrix



ROC-AUC Curves (One-vs-Rest)



วิเคราะห์: Class 26 & 30 F1 สูงสุด (0.94 / 0.90) | Class 28 & 29 F1 = 0.69 รูปร่างคล้ายกัน (ดู Confusion Matrix) | AUC ≈ 0.97 ทุกคลาส

สมาชิกในกลุ่ม

1) นาโอโตะ จูระ 1640312102

หน้าที่ : 20% ทำการ train และ evaluate โมเดลด้วย CNN และ วิเคราะห์ผลการทดลอง

2) พีรณย์ กันทะมา 1640701551

หน้าที่ : 20% ทำ front end และ admin page

3) ธนกร ปัญญาปัญญรัตน์ 1650708405

หน้าที่ : 20% ทำ เว็บแอปเก็บข้อมูล

4) อัสริยา ธรรภาพิสิฐ 1670701224

หน้าที่ : 20% ทำการ train และ evaluate โมเดลด้วย CRNN และ วิเคราะห์ผลการทดลอง

5) นรากร นิลรวม 1670704681

หน้าที่ : 20% ทำการ train และ evaluate โมเดล ด้วย CRNN และ วิเคราะห์ผลการทดลอง